

Fast and Focused: Interactive Repository-Level Fill-in-the-Middle Code Completion with Hybrid Structural Retrieval

Anonymous IJCAI submission

Abstract

Recent advances in Large Language Models (LLMs) have substantially improved code completion performance. However, most existing approaches remain confined to the file level and struggle in repository-level settings, where relevant context is distributed across multiple files and interconnected through structural dependencies. To address this challenge, we introduce ACAR, a repository-level Fill-In-the-Middle (FIM) code completion framework that emphasizes structural awareness and efficient context selection. ACAR combines hybrid retrieval by integrating lexical similarity with graph-based dependency signals, and maintains up-to-date structural information through lightweight synchronization with the Abstract Syntax Tree (AST) via the Language Server Protocol (LSP). To further reduce noise, the retrieved context is refined using AST-guided structural filtering prior to generation, which is particularly important for FIM scenarios that require precise alignment with both preceding and succeeding code. We evaluate ACAR on three representative benchmarks—RepoEval, ReccEval, and CrossCodeEval—spanning more than 3,000 real-world repositories across multiple programming languages, including Python, Java, TypeScript, and C#. Experimental results show that ACAR consistently outperforms strong repository-level baselines in completion accuracy and contextual alignment, while maintaining low-latency prompt construction prior to model inference through efficient retrieval and filtering suitable for interactive development environments. These findings demonstrate that coupling hybrid retrieval with structural awareness is an effective strategy for practical and high-quality repository-level FIM code completion. Code and evaluation scripts are available at <https://anonymous.4open.science/r/ACAR>.

1 Introduction

Large Language Models (LLMs) have rapidly become an integral component of modern software development, assisting

developers in tasks such as code completion, bug fixing, and automated documentation [Barke *et al.*, 2022]. By learning statistical regularities from massive code corpora, these models have enabled tools such as COPILOT¹ and CODEX² to accelerate development workflows, reduce syntactic errors, and improve overall productivity. Among these applications, code completion plays a particularly central role. It lowers cognitive load, facilitates rapid prototyping, and helps developers navigate unfamiliar or continuously evolving codebases more efficiently.

Early LLM-based code completion systems, including CODEX and CODEGEN [Nijkamp *et al.*, 2023], demonstrated strong performance in isolated coding scenarios. However, these approaches primarily operate in an *in-file* setting, where predictions are conditioned on the local context of a single source file. While effective for short snippets, this assumption breaks down in real-world software development, which is inherently modular. Modern software systems consist of multiple files, classes, and modules that interact through imports, shared data structures, and cross-file function calls. Such modularity introduces complex structural dependencies that static, file-level models struggle to capture, which limits their effectiveness in large and evolving codebases.

To better address this challenge, recent research has shifted toward repository-level code completion. Methods such as REPOCODER [Zhang *et al.*, 2023a], GRAPHCODER [Liu *et al.*, 2024], and REPOFORMER [Wu *et al.*, 2024] expand the effective context window through retrieval mechanisms that incorporate information from across the repository. Other approaches, including RLCODER [Wang *et al.*, 2024], further explore selective attention or reinforcement learning to improve retrieval efficiency. Despite these advances, existing repository-level methods continue to face two fundamental challenges. First, retrieved context is often noisy and contains irrelevant or outdated code due to limited **structural filtering**. Second, retrieval is typically performed over static repository snapshots, which constrains the ability to reflect **up-to-date structural changes** that arise during active development. As a result, the retrieved context may become misaligned with the current state of the codebase, reducing reliability in interactive development workflows.

¹<https://copilot.microsoft.com/>

²<https://openai.com/codex/>

In this paper, we present ACAR, a repository-level framework for *Fill-in-the-Middle* (FIM) code completion. FIM poses a distinct challenge compared to left-to-right completion, as the model must simultaneously reason over both preceding and succeeding context in order to generate a missing code segment that remains consistent with surrounding structure. Addressing this setting requires not only broader contextual coverage but also precise alignment with the structural state of the codebase. To this end, ACAR integrates three complementary components:

- **Hybrid retrieval**, which combines lexical similarity with graph-based dependency signals to capture both semantic relevance and structural relationships across the repository.
- **AST synchronization via LSP**, which maintains up-to-date structural information through lightweight synchronization with the *Abstract Syntax Tree* (AST) using the *Language Server Protocol* (LSP).
- **Structural filtering**, which refines retrieved candidates using AST guidance to prune irrelevant or outdated context prior to generation.

By tightly coupling retrieval with up-to-date structural information, ACAR enables precise and context-aware repository-level FIM completion while maintaining low-latency prompt construction suitable for interactive development environments. Rather than indiscriminately expanding the context window, ACAR focuses on selecting and refining structurally relevant information that aligns with the current state of the codebase. This design addresses both relevance and efficiency concerns in repository-level code completion.

Our main contributions are summarized as follows.

- We introduce ACAR, a repository-level FIM code completion framework that tightly couples hybrid retrieval with AST-guided structural filtering and lightweight structural synchronization.
- We propose an AST-aware hybrid retrieval strategy that jointly leverages textual similarity and graph-based dependency information to reduce noisy and stale context in repository-level FIM scenarios.
- We conduct extensive evaluations on three benchmarks, namely RepoEval, ReccEval, and CrossCodeEval, demonstrating that ACAR achieves state-of-the-art or competitive performance compared to strong repository-level baselines across multiple programming languages, while remaining suitable for interactive development settings.

2 Related Work

2.1 Modern Approaches for Code Completion

Research on code completion has evolved along three levels of contextual scope: *in-file*, *cross-file*, and *repository-level*. Each level progressively incorporates richer semantic and structural information to improve completion accuracy.

In-file context. With the advent of Transformer architectures, substantial effort has been devoted to pretraining language models on large-scale code corpora [Feng *et al.*, 2020;

Ahmad *et al.*, 2021; Ding *et al.*, 2022]. These models underpin many modern code intelligence systems, particularly for code generation and completion [Husain *et al.*, 2020]. Representative examples include open-source autoregressive models [Black *et al.*, 2022] and Incoder [Fried *et al.*, 2023], which achieve strong performance using only local, file-level context. However, such approaches are inherently limited when code semantics extend beyond a single file.

Cross-file context. To move beyond local context, cross-file approaches exploit semantic signals across multiple files within a project. Early systems such as CODEFINDER supported example-based programming through similarity-based retrieval [Henninger, 1991]. Subsequent work incorporated static analysis and learning over near-duplicate code segments to assist in completing variable declarations and method bodies [Hill and Rideout, 2004]. Despite these advances, cross-file methods remain relatively underexplored due to challenges in dependency resolution, scalability, and maintaining relevance in large codebases.

Repository-level context. More recent research focuses on repository-wide reasoning, where models capture semantic and structural relationships across interdependent files. Representative approaches include similarity-based retrieval methods such as ReACC [Lu *et al.*, 2022], hybrid retrieval-generation frameworks such as RepoCoder [Zhang *et al.*, 2023b] and CoCoMIC [Ding *et al.*, 2023b], as well as more structurally informed methods that leverage program graphs and retrieval-augmented generation. For example, DRACO constructs semantic graphs via dataflow analysis [Cheng *et al.*, 2024], while RepoFormer employs selective *Retrieval-Augmented Generation* (RAG) with self-supervised filtering to reduce redundant context [Wu *et al.*, 2024].

Despite these advances, several fundamental limitations remain. Most repository-level methods rely on textual retrieval or static program analysis, which can introduce noisy or incomplete context in large, modular repositories. Retrieval is typically performed over static repository snapshots, limiting the ability to reflect structural updates during active development. In addition, existing approaches often adopt homogeneous retrieval strategies, such as purely text-based or purely graph-based retrieval, which restricts their ability to jointly capture semantic relevance and structural dependencies. These limitations are especially critical in FIM code completion, where precise alignment with both preceding and succeeding context is required. Motivated by these gaps, we propose ACAR, a repository-level FIM code completion framework that integrates hybrid retrieval, real-time AST synchronization, and AST-guided structural filtering to provide accurate, up-to-date, and structurally aligned context.

2.2 Applications of RAG

RAG enhances large language models with external knowledge sources, improving factuality and robustness [Gao *et al.*, 2024; Mallen *et al.*, 2023]. Retrieval strategies range from keyword-based methods such as BM25 [Robertson and Zaragoza, 2009], to dense embedding-based retrieval [Lewis *et al.*, 2021], as well as interleaved and iterative techniques for complex reasoning [Trivedi *et al.*, 2023]. More recently,

graph-structured representations have motivated Graph-based RAG, which leverages graph topology to improve retrieval precision and contextual coherence [Hu *et al.*, 2024]. These techniques have also been applied to code intelligence tasks, including code generation from natural language descriptions [Li *et al.*, 2024] and program summarization [Zhou *et al.*, 2023]. While effective, many RAG-based systems face trade-offs between retrieval latency and generation quality. In contrast, our approach emphasizes both efficiency and accuracy, enabling low-latency retrieval and high-quality code completion suitable for interactive development settings.

3 The ACAR Framework

Overall Framework. To enable accurate and responsive repository-level code completion, we propose ACAR, a structurally-aware framework composed of four stages: (i) repository pre-processing, (ii) context retrieval, (iii) prompt construction, and (iv) post-processing. Figure 1 illustrates the overall architecture. The central idea of ACAR is to construct a structured representation of the repository that allows the completion model to exploit both local lexical similarity and global structural dependencies. To this end, ACAR integrates two complementary retrieval mechanisms, text-based and graph-based, which jointly inform the prompt construction stage. The framework is model-agnostic and can be instantiated with any backbone code generation model, making it suitable for diverse deployment scenarios.

3.1 Repository Pre-processing

We formalize code completion as a mapping

$$Y = M(X),$$

where X denotes an incomplete code snippet and Y the generated completion. To enrich the input context, ACAR constructs a retrieval database from repository-level information. Specifically, the repository is partitioned into a corpus of code segments $C_{\text{repo}} = \{c_1, c_2, \dots, c_N\}$, and simultaneously represented as a directed code graph derived from ASTs, $G_{\text{repo}} = (V, E)$, where nodes correspond to code entities such as modules, classes, functions, and variables. This dual representation enables both lexical and structural queries over the repository.

3.2 Context Retrieval

Given an incomplete snippet X , ACAR performs text-based and graph-based retrieval in parallel:

$$C_{\text{text}} = R_{\text{text}}(C_{\text{repo}}, X), \quad C_{\text{graph}} = R_{\text{graph}}(G_{\text{repo}}, X).$$

The retrieved contexts are later fused and ranked before being incorporated into the model prompt.

Text-based Retrieval. Text-based retrieval follows a sliding-window strategy with Jaccard similarity. Each candidate window S_m of size w is tokenized and compared with the query window S_t derived from the tail of X :

$$J(S_t, S_m) = \frac{|S_t \cap S_m|}{|S_t \cup S_m|}.$$

The top- k windows with the highest similarity scores are selected to form C_{text} . This component captures surface-level lexical overlap that often reflects local semantic relevance.

Graph-based Retrieval. To capture structural dependencies, ACAR constructs a repository-level code graph using ASTs and symbol information obtained via the LSP. Edges encode containment, dependency, and import relations:

$$E = \{(u, r, v) \mid u, v \in V, r \in \{\text{contains, depends, imports}\}\}.$$

Given the node v_0 corresponding to snippet X , graph-based retrieval performs a bounded neighborhood traversal to obtain structurally related entities: $C_{\text{graph}} = \text{DFS}(v_0, s)$, where the traversal depth s controls the retrieval scope. Integration with LSP ensures that the graph remains synchronized with repository edits, allowing ACAR to reflect up-to-date structural information.

Context Fusion and Ranking. The outputs of text-based and graph-based retrieval are merged using *Reciprocal Rank Fusion* (RRF). For each candidate line L , the fused score is computed as

$$\text{RRF}(L) = \sum_{Q_j \in \mathcal{Q}} \frac{1}{\text{rank}_{Q_j}(L) + c},$$

where $\mathcal{Q}$ denotes the set of retrieval queries and c is a damping constant. This fusion strategy ensures that both lexical similarity and structural relevance jointly influence the final ranking.

3.3 Prompt Construction.

ACAR adopts a FIM prompting scheme to incorporate repository-level context into generation. Each prompt is constructed as

$$\text{prefix} [\text{MASK}] \text{suffix} + C_{\text{text}} + C_{\text{graph}},$$

where the retrieved contexts are concatenated with the standard prefix and suffix. This design allows the model to reason over both local editing context and relevant repository-level signals, while remaining compatible with existing LLM backbones such as Qwen2.5-Coder and DeepSeek-Coder.

3.4 Post-processing

Raw model outputs may contain redundant or structurally misaligned code. ACAR applies a lightweight AST-guided pruning procedure to ensure syntactic alignment with the surrounding code. Starting from the insertion node n_0 , the algorithm selects the highest ancestor node that satisfies a line-alignment criterion, preventing duplication and preserving structural validity.

This post-processing step filters extraneous code, preserves repository style, and produces clean, evaluation-ready completions. Empirically, we find that AST-guided pruning substantially improves the usability of generated code, consistent with prior observations [Gong *et al.*, 2024].

Design Rationale. The design of ACAR balances efficiency, accuracy, and robustness in repository-level code completion. Repository pre-processing enables complementary lexical and structural views. Parallel retrieval and rank fusion reduce noise while preserving relevant signals. FIM-based prompt construction conditions generation on local and

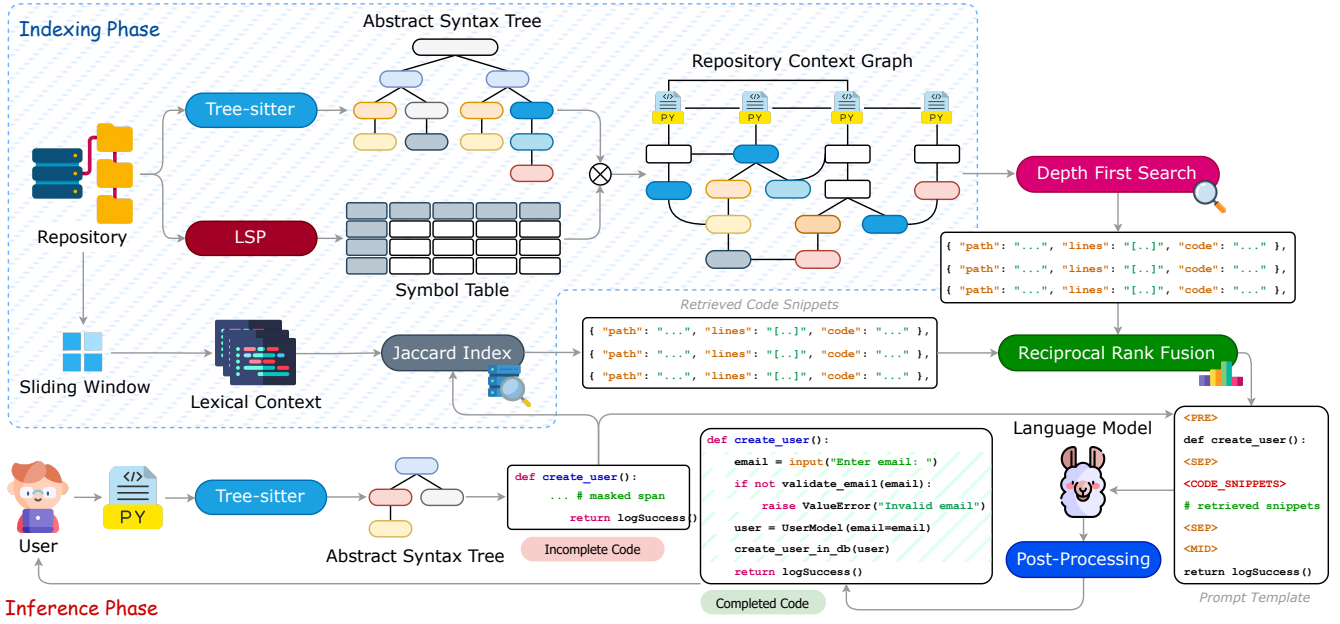


Figure 1: Overview of the structurally-aware repository-level FIM pipeline in ACAR. The system operates in two phases: indexing (repository analysis and retrieval) and inference (prompt construction and completion).

Algorithm 1 Line-Aligned Ancestor Selection

```

1: Input: Initial node  $n_0$ ; functions  $parent(\cdot)$  and  $line(\cdot)$ ; predicate  $BEGINSWITHOPENDELIMITER()$ .
2: Output: Highest ancestor  $n$  satisfying the alignment criterion.
3:  $n \leftarrow n_0$ 
4: while  $n \neq \text{nil}$  and  $parent(n) \neq \text{nil}$  do
5:    $p \leftarrow parent(n)$ 
6:   if  $line(p) = line(n_0)$  then
7:      $n \leftarrow p$ 
8:   else if  $line(p) = line(n_0) - 1$  and  $BEGINSWITHOPENDELIMITER()$  then
9:      $n \leftarrow p$ 
10:  else
11:    break
12:  end if
13: end while
14: return  $n$ 

```

global context. Finally, AST-guided post-processing ensures structural validity. Together, these components form a cohesive framework in which retrieval and generation are tightly integrated, enabling reliable repository-level FIM code completion across tasks, languages, and backbone models.

4 Experiments

In this section, we conduct a series of experiments to systematically evaluate the effectiveness of ACAR. Specifically, we address the following research questions:

RQ1 How does ACAR perform across three repository-level code completion benchmarks?

RQ2 How efficient and robust is ACAR under realistic development constraints?

RQ3 How well does ACAR generalize across different programming languages?

RQ4 How do individual modules of ACAR contribute to overall performance?

4.1 Study Setup

Datasets. We evaluate ACAR on three widely used benchmarks that collectively cover large-scale, repository-level, and multilingual code completion: ReccEval [Cheng *et al.*, 2024], RepoEval [Zhang *et al.*, 2023b], and CrossCodeEval [Ding *et al.*, 2023a]. ReccEval consists of thousands of real-world Python repositories curated from PyPI, reflecting large-scale repository completion scenarios. RepoEval focuses on repository-level completion with line-, API-, and function-level tasks, where correctness is validated through project-specific unit tests. CrossCodeEval extends evaluation to four programming languages (Python, Java, TypeScript, and C#), enabling assessment of context-dependent completion across diverse syntactic and structural settings.

Baselines. We compare ACAR against five repository-level code completion methods spanning diverse retrieval and modeling paradigms: DraCo [Cheng *et al.*, 2024] (dataflow-based structural analysis), CoCoMic [Ding *et al.*, 2023b] (static cross-file clone retrieval), RepoCoder [Zhang *et al.*, 2023b] (iterative retrieval and generation), GraphCoder [Liu *et al.*, 2024] (explicit graph-based structural modeling), and RepoFormer [Wu *et al.*, 2024] (selective RAG with self-supervised filtering). Together, these baselines cover text-, graph-, hybrid, and iterative retrieval strategies. For all

baselines, we follow the original implementations and hyperparameter settings reported in their papers, and evaluate all methods using the same Qwen backbone and decoding configuration to ensure a fair comparison.

Evaluation Metrics. We evaluate code completion quality using two complementary metrics. *Exact Match* (EM) measures the percentage of predictions that exactly match the ground-truth snippet after canonical formatting, and is most informative for short completions (e.g., line-level). *Edit Similarity* (ES) computes the normalized Levenshtein similarity between prediction and reference, ranging from 0 to 1, and captures near-miss outputs. ES is better suited for longer completions (e.g., function-level), where semantically correct results may differ lexically.

ACAR Configuration. Table 1 summarizes the inference hyperparameters used in ACAR. For FIM tasks, generation terminates upon encountering predefined special tokens. All experiments are conducted without quantization to preserve model fidelity.

Table 1: Inference hyperparameters for ACAR.

General	
Token_in	8000
Token_out	48
Temperature	0.2
Top-k	5
Stop sequences	{< fim_prefix >, < fim_suffix >, < fim_middle >, < endoftext >}
Quantization	None
Task-specific	
Sliding window S_m	50
Step size S_s	10
Graph depth	2

Machine Configuration All experiments were conducted on a server equipped with an AMD EPYC 7V13 64-core processor (24 cores available, 1 thread per core, base clock 2.5 GHz), an NVIDIA A100 GPU with 80 GB memory, and 512 GB of system RAM. The software environment was based on Ubuntu 20.04.6 LTS (64-bit).

4.2 Performance Evaluation (RQ1)

Tables 2, 3, and 4 summarize the performance of ACAR on RepoEval, ReccEval, and CrossCodeEval, respectively. Across all three benchmarks, ACAR consistently achieves the strongest results under both Qwen2.5-Coder-0.5B and Qwen2.5-Coder-3B, outperforming representative repository-level baselines on both EM and ES. On RepoEval, ACAR yields substantial gains across line-, API-, and function-level completion, demonstrating robustness across different completion granularities. On ReccEval, which evaluates large-scale Python repositories, ACAR establishes clear margins over prior methods, particularly with the 3B backbone, indicating effective exploitation of repository context. On the multilingual CrossCodeEval benchmark, ACAR maintains strong and consistent improvements across Python, Java, TypeScript, and C#, confirming its ability to generalize across diverse programming languages. Overall, these re-

sults validate that hybrid retrieval combined with AST-guided structural filtering enables accurate and reliable repository-level Fill-in-the-Middle code completion across tasks, scales, and languages.

4.3 Practical Applicability (RQ2)

Figure 2 and Figure 3 summarize the practical applicability of ACAR in terms of efficiency and robustness. As shown in Figure 2, ACAR consistently achieves substantially lower prompt generation latency than all baselines across ReccEval, RepoEval, and CrossCodeEval, reducing prompt construction time to under 100 ms while competing methods require several hundred milliseconds or even multiple seconds. This efficiency gain demonstrates that hybrid retrieval and structural filtering can be integrated without incurring prohibitive overhead, making ACAR suitable for interactive development workflows. Figure 3 further shows that ACAR maintains stable completion quality across different backbone models, ranging from lightweight (Qwen2.5-Coder-0.5B) to larger architectures (DeepSeek-Coder-6.7B), with relatively small variance in normalized EM and ES scores. Together, these results indicate that ACAR is both responsive and robust, delivering reliable repository-level code completion under realistic deployment constraints.

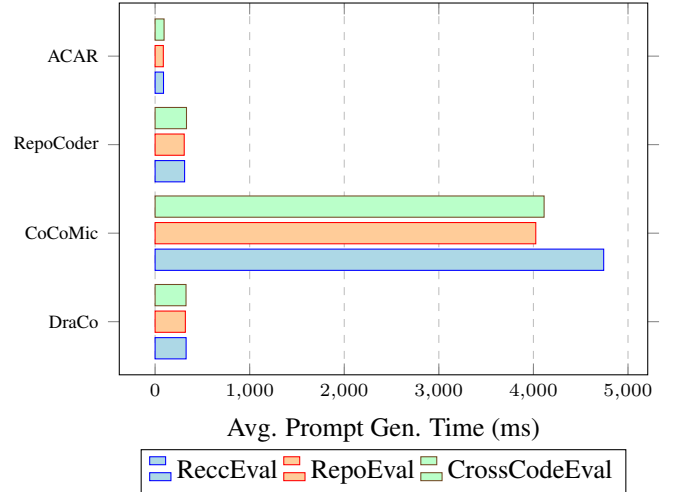


Figure 2: Average prompt generation time (ms) across datasets. ACAR consistently achieves much lower latency compared to baseline methods.

4.4 Multilingual Generalization (RQ3)

Table 5 reports the multilingual results on CrossCodeEval. Across all four languages, ACAR consistently achieves the highest or near-highest EM and ES scores under both backbones, demonstrating strong generalization beyond Python-centric repositories. With Qwen2.5-Coder-3B, ACAR attains an average EM of 32.86 and ES of 74.34, substantially outperforming RepoCoder and CoCoMic, and surpassing graph-based baselines such as GraphCoder and RepoFormer across most languages. Notably, the gains are consistent across syntactically diverse languages, including statically typed (Java,

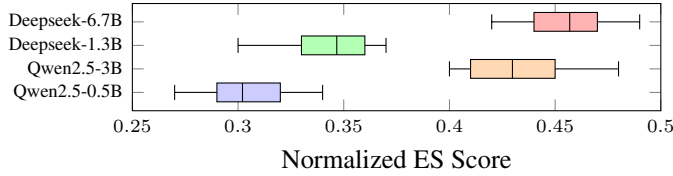
Table 2: RepoEval results (%) under different backbones.

Method	Qwen2.5-Coder-0.5B						Qwen2.5-Coder-3B					
	Line		API		Function		Line		API		Function	
	EM	ES	EM	ES	EM	ES	EM	ES	EM	ES	EM	ES
DraCo	18.75	42.38	17.69	45.18	2.22	32.17	34.92	55.95	27.95	57.72	2.64	34.83
CoCoMic	23.51	44.38	18.16	44.30	1.47	30.04	34.48	53.90	<u>28.16</u>	55.23	2.33	34.23
RepoCoder	24.58	<u>58.56</u>	16.78	48.78	2.12	32.78	38.03	<u>67.43</u>	28.32	56.23	2.32	34.38
GraphCoder	26.91	50.88	17.94	53.47	2.86	46.73	32.97	61.48	25.86	61.72	5.29	54.61
Repoformer	<u>28.12</u>	52.34	<u>18.11</u>	55.45	<u>3.83</u>	48.92	34.68	64.27	26.82	62.11	<u>6.14</u>	56.20
ACAR (text-only)	24.44	53.64	17.43	<u>58.24</u>	3.65	<u>50.60</u>	37.27	63.84	26.44	<u>67.03</u>	6.11	<u>57.80</u>
ACAR (graph-only)	23.74	51.64	16.93	57.74	3.35	50.30	36.77	62.34	25.94	66.53	5.81	57.30
ACAR (w/o post.)	21.38	44.29	14.46	46.51	2.42	48.89	33.62	54.65	22.84	54.47	4.62	54.99
ACAR	33.62	54.65	19.96	59.57	5.27	51.71	<u>37.50</u>	69.52	29.04	68.43	8.35	59.60

Table 3: Results and ablation study of ACAR on the ReccEval dataset (%).

Method	Qwen2.5-Coder-0.5B		Qwen2.5-Coder-3B	
	EM	ES	EM	ES
DraCo	26.25	60.95	33.27	68.38
CoCoMic	14.43	56.36	20.85	62.59
RepoCoder	24.57	62.82	30.10	66.94
GraphCoder	25.83	62.94	33.71	68.02
Repoformer	<u>27.97</u>	63.41	36.25	<u>69.20</u>
ACAR (text-only retrieval + postproc.)	25.51	51.73	<u>39.24</u>	64.82
ACAR (graph-only retrieval + postproc.)	24.81	50.43	<u>38.49</u>	63.37
ACAR (w/o post processing)	19.41	39.83	33.99	54.28
ACAR	30.21	61.03	42.98	72.46

(a) ReccEval: Edit Similarity (ES)



(b) ReccEval: Exact Match (EM)

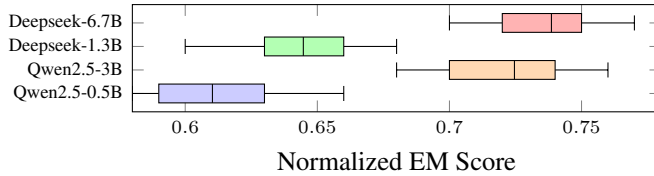


Figure 3: ReccEval performance per model. (a) Edit Similarity (ES). (b) Exact Match (EM).

dencies rather than relying on surface-level lexical cues. In contrast, variants without post-processing show degraded performance, confirming the importance of structural alignment for multilingual repository-level FIM completion. Overall, these results validate that ACAR generalizes robustly across heterogeneous programming environments.

4.5 Module Contribution (RQ4)

Ablation results on ReccEval, RepoEval, and CrossCodeEval demonstrate that ACAR’s performance gains arise from the complementary contributions of all modules. Text-only retrieval improves lexical alignment but struggles with long-range dependencies, while graph-only retrieval captures structural relations yet lacks sufficient surface-level grounding, leading to inferior EM and ES when used in isolation. Removing post-processing causes a consistent performance drop across all benchmarks, as raw generations often contain redundant or structurally misaligned code, highlighting the necessity of AST-guided refinement. The full ACAR framework, which integrates text-based retrieval, graph-based retrieval, and post-processing, consistently achieves the best results, confirming that no single component is sufficient on its own. These findings validate that ACAR’s hybrid design is essential for robust and accurate repository-level FIM code completion.

C#) and structurally flexible languages (Python, TypeScript), indicating that ACAR’s hybrid retrieval and AST-guided filtering effectively capture language-agnostic structural depen-

Table 4: Results and ablation study of ACAR on the CrossCodeEval dataset (%).

Method	Qwen2.5-Coder-0.5B		Qwen2.5-Coder-3B	
	EM	ES	EM	ES
DraCo	17.61	64.56	22.97	69.75
CoCoMic	9.85	60.45	13.49	63.98
RepoCoder	12.97	63.93	16.85	67.54
GraphCoder	16.72	65.01	23.94	69.02
Repoformer	19.42	66.17	28.48	71.23
ACAR (text-only retrieval + postproc.)	18.34	60.28	30.73	70.87
ACAR (graph-only retrieval + postproc.)	17.64	58.98	30.43	69.37
ACAR (w/o post processing)	14.15	51.58	28.59	65.40
ACAR	22.13	67.38	32.86	74.34

Table 5: CrossCodeEval results per language. Exact Match (EM) and Edit Similarity (ES) are reported for Qwen2.5-Coder-0.5B and Qwen2.5-Coder-3B backbones. DraCo and CoCoMic are shown without language-level breakdown.

Method	Backbone	Python		Java		TypeScript		C#		Avg.	
		EM	ES	EM	ES	EM	ES	EM	ES	EM	ES
DraCo	0.5B	20.15	66.42	15.87	63.18	16.23	63.95	18.19	64.69	17.61	64.56
	3B	26.38	71.54	20.65	68.43	21.12	69.28	23.73	69.75	22.97	69.75
CoCoMic	0.5B	11.86	62.34	8.47	59.28	8.92	59.67	10.15	60.51	9.85	60.45
	3B	16.24	65.87	11.58	62.76	12.03	63.14	14.11	64.15	13.49	63.98
RepoCoder	0.5B	12.97	63.93	11.24	62.47	11.89	63.12	15.78	66.20	12.97	63.93
	3B	16.85	67.54	14.93	66.18	15.42	66.83	20.20	69.61	16.85	67.54
GraphCoder	0.5B	17.86	66.02	14.91	64.21	16.04	64.88	18.07	65.94	16.72	65.01
	3B	28.71	71.02	22.68	68.94	23.41	69.21	20.95	66.91	23.94	69.02
Repoformer	0.5B	24.83	69.37	15.94	64.08	18.27	65.42	18.56	65.53	19.40	66.10
	3B	34.17	73.08	25.36	70.19	26.08	70.61	27.98	71.93	28.40	71.20
ACAR (w/o pp)	0.5B	13.43	44.14	12.16	52.14	11.59	53.44	19.40	56.62	14.15	51.58
	3B	29.76	59.33	25.20	66.23	25.00	67.65	34.39	68.39	28.59	65.40
ACAR (full)	0.5B	28.74	75.56	17.63	65.99	20.67	63.37	21.49	64.61	22.13	67.38
	3B	39.89	75.56	29.31	73.82	26.43	73.33	35.80	74.66	32.86	74.34

5 Conclusion

We presented ACAR, a structurally-aware framework for repository-level Fill-in-the-Middle code completion. Unlike approaches that rely on retrieving increasingly large amounts of context, ACAR emphasizes retrieving *structurally aligned* context by jointly leveraging lexical similarity and graph-based dependency signals. The framework maintains an up-to-date view of the repository through lightweight AST synchronization via the Language Server Protocol, and applies AST-guided post-processing to filter redundant or misaligned generations.

Extensive experiments on RepoEval, ReccEval, and CrossCodeEval demonstrate that ACAR consistently improves completion quality across tasks, programming languages, and model backbones. The gains are particularly pronounced in scenarios where the missing span depends on cross-file symbols or non-local dependencies, which are difficult to recover

from in-file context alone. Ablation studies further reveal the complementary roles of ACAR’s components: text-based retrieval provides strong lexical anchoring, graph-based retrieval supplies dependency-aware signals beyond the current file, and AST-guided post-processing materially improves structural compatibility with surrounding code in Fill-in-the-Middle settings.

From a systems perspective, ACAR is designed with interactive development in mind. Its prompt construction pipeline remains lightweight, and LSP-based synchronization ensures robustness to ongoing code edits, helping bridge the gap between offline repository-level evaluation and real-world editor usage. Overall, our findings indicate that coupling retrieval with structural awareness is a practical and effective strategy for reliable repository-level FIM completion, and motivate future work on richer program representations and tighter integration with developer workflows.

References

- [Ahmad *et al.*, 2021] Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. Unified pre-training for program understanding and generation. In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou, editors, *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2655–2668, Online, June 2021. Association for Computational Linguistics.
- [Barke *et al.*, 2022] Shraddha Barke, Michael B. James, and Nadia Polikarpova. Grounded copilot: How programmers interact with code-generating models, 2022.
- [Black *et al.*, 2022] Sidney Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, Michael Pieler, Usven Sai Prashanth, Shivanshu Purohit, Laria Reynolds, Jonathan Tow, Ben Wang, and Samuel Weinbach. GPT-NeoX-20B: An open-source autoregressive language model. In Angela Fan, Suzana Ilic, Thomas Wolf, and Matthias Gallé, editors, *Proceedings of Big-Science Episode #5 – Workshop on Challenges & Perspectives in Creating Large Language Models*, pages 95–136, virtual+Dublin, May 2022. Association for Computational Linguistics.
- [Cheng *et al.*, 2024] Wei Cheng, Yuhan Wu, and Wei Hu. Dataflow-guided retrieval augmentation for repository-level code completion. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7957–7977, Bangkok, Thailand, August 2024. Association for Computational Linguistics.
- [Ding *et al.*, 2022] Yangruibo Ding, Luca Buratti, Saurabh Pujar, Alessandro Morari, Baishakhi Ray, and Saikat Chakraborty. Towards learning (dis)-similarity of source code from program contrasts. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio, editors, *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6300–6312, Dublin, Ireland, May 2022. Association for Computational Linguistics.
- [Ding *et al.*, 2023a] Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023.
- [Ding *et al.*, 2023b] Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. Co-comic: Code completion by jointly modeling in-file and cross-file context, 2023.
- [Feng *et al.*, 2020] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A pre-trained model for programming and natural languages. In Trevor Cohn, Yulan He, and Yang Liu, editors, *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, Online, November 2020. Association for Computational Linguistics.
- [Fried *et al.*, 2023] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen tau Yih, Luke Zettlemoyer, and Mike Lewis. InCoder: A generative model for code infilling and synthesis, 2023.
- [Gao *et al.*, 2024] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey, 2024.
- [Gong *et al.*, 2024] Linyuan Gong, Sida Wang, Mostafa Elhoushi, and Alvin Cheung. Evaluation of llms on syntax-aware code fill-in-the-middle tasks. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- [Henninger, 1991] Scott Henninger. Retrieving software objects in an example-based programming environment. In *Proceedings of the 14th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR ’91*, page 251–260, New York, NY, USA, 1991. Association for Computing Machinery.
- [Hill and Rideout, 2004] R. Hill and J. Rideout. Automatic method completion. In *Proceedings. 19th International Conference on Automated Software Engineering, 2004.*, pages 228–235, 2004.
- [Hu *et al.*, 2024] Yuntong Hu, Zhihan Lei, Zheng Zhang, Bo Pan, Chen Ling, and Liang Zhao. Grag: Graph retrieval-augmented generation, 2024.
- [Husain *et al.*, 2020] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. Codesearchnet challenge: Evaluating the state of semantic code search, 2020.
- [Lewis *et al.*, 2021] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.
- [Li *et al.*, 2024] Jia Li, Yunfei Zhao, Yongmin Li, Ge Li, and Zhi Jin. Acecoder: An effective prompting technique specialized in code generation. *ACM Trans. Softw. Eng. Methodol.*, 33(8), November 2024.
- [Liu *et al.*, 2024] Wei Liu, Ailun Yu, Daoguang Zan, Bo Shen, Wei Zhang, Haiyan Zhao, Zhi Jin, and Qianxiang Wang. Graphcoder: Enhancing repository-level code completion via coarse-to-fine retrieval based on code context graph. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*,

- ASE '24, page 570–581, New York, NY, USA, 2024. Association for Computing Machinery.
- [Lu *et al.*, 2022] Shuai Lu, Nan Duan, Hojae Han, Daya Guo, Seung-won Hwang, and Alexey Svyatkovskiy. ReACC: A retrieval-augmented code completion framework. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio, editors, *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6227–6240, Dublin, Ireland, May 2022. Association for Computational Linguistics.
- [Mallen *et al.*, 2023] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9802–9822, Toronto, Canada, July 2023. Association for Computational Linguistics.
- [Nijkamp *et al.*, 2023] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. Codegen: An open large language model for code with multi-turn program synthesis, 2023.
- [Robertson and Zaragoza, 2009] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Found. Trends Inf. Retr.*, 3(4):333–389, April 2009.
- [Trivedi *et al.*, 2023] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions, 2023.
- [Wang *et al.*, 2024] Yanlin Wang, Yanli Wang, Daya Guo, Jiachi Chen, Ruikai Zhang, Yuchi Ma, and Zibin Zheng. Rl-coder: Reinforcement learning for repository-level code completion, 2024.
- [Wu *et al.*, 2024] Di Wu, Wasi Uddin Ahmad, Dejiao Zhang, Murali Krishna Ramanathan, and Xiaofei Ma. Repoformer: selective retrieval for repository-level code completion. In *Proceedings of the 41st International Conference on Machine Learning, ICML'24*. JMLR.org, 2024.
- [Zhang *et al.*, 2023a] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2471–2484, Singapore, December 2023. Association for Computational Linguistics.
- [Zhang *et al.*, 2023b] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. Repocoder: Repository-level code completion through iterative retrieval and generation, 2023.
- [Zhou *et al.*, 2023] Ziyi Zhou, Huiqun Yu, Guisheng Fan, Zijie Huang, and Kang Yang. Towards retrieval-based neural code summarization: A meta-learning approach. *IEEE Transactions on Software Engineering*, 49(4):3008–3031, 2023.